To-Do Task Manager – Project Report

Introduction

This report presents the development of the **Ultimate To-Do Task Manager**, a Transformers-themed, single-page web application built for the Software and DevOps course.

The goal of this project was to design and implement a minimal software application with **CRUD functionality, persistent storage, and REST API endpoints**, while also documenting the full **Software Development Life Cycle (SDLC)**.

The application combines a **Flask backend** with **JSON file persistence** and a **vanilla HTML/CSS/JS frontend**. In addition to meeting all core requirements, extra features such as offline support, achievements, and animations were added to enhance usability and user engagement.

Chosen SDLC Model

The **Agile/Iterative SDLC model** was chosen.

Unlike Waterfall, which forces sequential completion of phases, Agile allowed for **incremental development**:

- A working prototype was quickly deployed and improved iteratively.
- Backend CRUD operations were tested early before frontend integration.
- Extra features (offline mode, achievements, stats HUD) were added in later iterations without disrupting the core.

This iterative approach reflects modern **DevOps practices**, where continuous integration, rapid feedback, and incremental scaling are central.

SDLC Documentation

Planning

Objective: Build a task manager with CRUD, persistence, REST API, and optional UI.
Rationale: The To-Do Manager was chosen because it maps naturally to CRUD operations,

is easy to test, and provides an expandable base for DevOps.
Scope: Minimal but functional app → then enhanced with gamified features (Transformers theme, achievements, offline support).

Requirements

Functional Requirements:

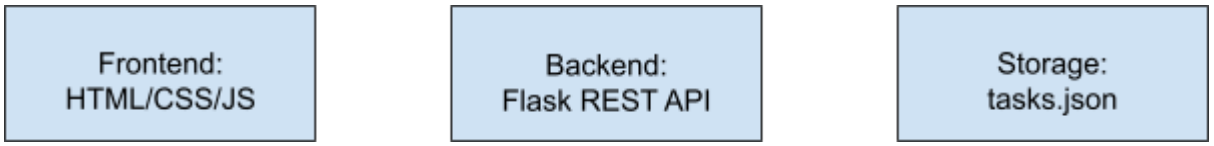
- Create, read, update, delete tasks.
- Store tasks persistently in tasks.json.
- Expose RESTful endpoints (/tasks).
- Frontend UI to interact with tasks.
- Inline editing, status toggling, filtering, and bulk operations.
- Achievements system unlocking characters when milestones are reached.

Non-Functional Requirements:

- Modular, readable code.
- Lightweight dependencies.
- Offline-first approach: cached data + queued operations when backend is unreachable.
- Portable and easy to run on local or LAN setup.

Design

The app follows a **3-tier architecture**:



- **Frontend:** Single-page app, animated Transformers theme, inline editing.
- **Backend:** Flask server (app.py), modular task logic (task.py, task_manager.py).

- **Storage:** JSON file persistence with immediate writes.

Additional design considerations:

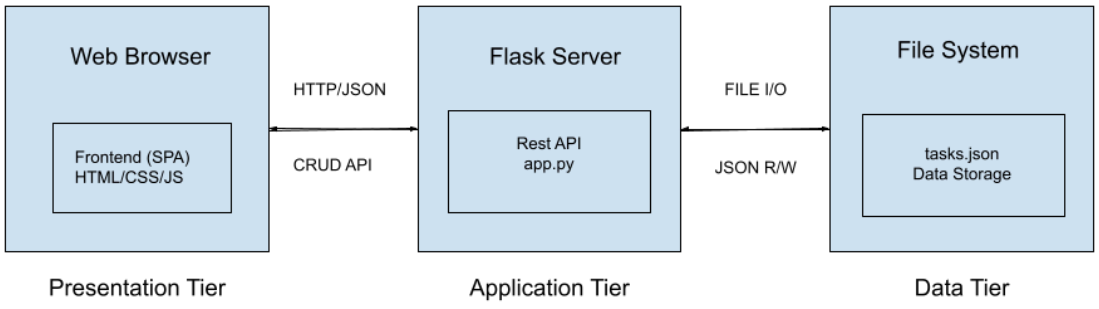
- **Offline-first:** Local caching and retry mechanisms.
- **Achievements system:** Unlocks characters after completing milestones.
- **Stats HUD:** Displays counts of completed, pending, total tasks.

Development

- **Backend:** Flask server exposing CRUD endpoints. Cross-Origin Resource Sharing (CORS) enabled.
- **Frontend:** Vanilla HTML/JS app consuming REST endpoints via fetch calls.
- **Storage:** JSON file (tasks.json) updated on every CRUD operation.
- **Version Control:** Individual Git repository with multiple commits documenting each milestone.
- **Extra features:** Offline mode, localStorage persistence, Transformers theme with animations and sound effects.

Architecture Overview

Architecture Diagram



Explanation

- **Presentation Tier (Browser):** User-facing UI, single-page design with filters, stats, and achievements.
- **Application Tier (Flask):** REST API with routes for CRUD operations.
- **Data Tier (File System):** tasks.json storing tasks with immediate persistence.

Reflection: DevOps Adaptation

This project is designed for easy integration into **DevOps pipelines**:

- **Continuous Integration:** CRUD tests can be automated to run on every commit.
- **Continuous Deployment:** Application can be containerized with Docker for deployment on any server or cloud provider.
- **Scalability:** Replace tasks.json with a database (SQLite, PostgreSQL, MongoDB) as user load increases.
- **Monitoring:** Logs and metrics could be captured with tools like Prometheus or ELK stack.
- **Automation:** GitHub Actions or Jenkins can handle linting, testing, and deployments.

By starting simple and modular, the app is highly adaptable to CI/CD, containerization, and cloud scaling.

Conclusion

The **Ultimate To-Do Task Manager** meets and exceeds the assignment requirements:

- **CRUD:** Full task lifecycle management.
- **Persistence:** JSON file storage with immediate writes.
- **API + UI:** REST backend with a themed, responsive frontend.

Through the **Agile SDLC model**, the project was developed incrementally, enabling rapid prototyping and later enhancements. Its **modular architecture** and **offline-first design** make it both robust and adaptable.

Finally, this project demonstrates how even a simple coursework app can be prepared for **scalability and DevOps-readiness**, serving as both a practical tool and a learning exercise in software engineering principles.

Use of AI

AI was leveraged selectively throughout this project, but the core implementation, logic, and backbone were entirely my work. Its role was primarily to refine clarity, structure, and presentation, therefore, helping me make the project cleaner, more polished, and easier to understand without replacing my own contribution.